



MALTEPE UNIVERSITY

Faculty of Engineering and Natural Sciences - Software Engineering

SE 342

Software Validation and Testing

Instructor: Dr. Emre Athier Olca

Prepared by:

22 07 06 059 - Ceren Yaşar

22 07 06 028 - Mehmet Şenadlı

22 07 06 018 - Bilal Çiftçi

Table of Contents

Software Requirements Specification: Exam Security System	3
1. Introduction	3
1.1 Project Purpose	3
1.2 Scope	3
1.3 How the System Works?	3
1.4 Why Did We Choose This Method?	4
2. Requirements	4
2.1. Business Rules & Validations (BRV)	4
2.1.1 Identity & Verification Rules (ML Logic).....	4
2.1.2 Seating & Compliance Rules	5
2.1.3 Role-Based Access Rules (RBAC)	5
2.1.4 Data Validation Rules (VR)	5
2.2 Functional Requirements (FR)	5
2.2 Non-Functional Requirements (NFR)	6
4. System Design	7
4.1 ERD (Entity Relationship Diagram)	7
4.2 Database Schema Design (PostgreSQL)	7
4.3 UML Diagrams	9
4.3.1 Use Case Diagram	9
4.3.2 Sequence Diagrams	9
4.3.3 Activity Diagram (Bonus)	11
5. Project Management & Methodology	11
5.1 Project Tracking with JIRA	11
5.2 Sprint Planning & Epic Roadmap	11
5.2.1 Sprint 1: Infrastructure & Analysis (Epic 1).....	12
5.2.2 Sprint 2: Core Development & Database (Epic 2)	12
5.2.3 Sprint 3: Check-in Workflow & ML Integration (Epic 3).....	12
5.2.4 Sprint 4: Reporting, Testing & Validation (Epic 4).....	12
6. Testing & Validation	12
6.1 Test Strategy and Objectives	12
6.2 Test Cases and Procedures	13
7. System Architecture & Technology Stack	15
7.1 Technology Stack	15
7.1.1 Project Management & Collaboration	15
7.1.2 Backend & AI Engine (Python)	16
7.1.3 Computer Vision & ML Integration	16
7.1.4 Frontend Framework (Vanilla JS & Modern CSS)	16
7.1.5 Database Management	16
7.2 Version Control Strategy	16
8. Unit Testing	17
9. Final System Screenshots	22

Software Requirements Specification: Exam Security System

🔗 For detailed tasks and progress, see the Jira board:

<https://exam-security-system.atlassian.net/jira/software/projects/SCRUM/boards/1/backlog>

🔗 For source code and repository structure, see the GitHub project:

<https://github.com/cerenyasarr/SE342-ExamSecuritySystem>

1. Introduction

1.1 Project Purpose

The purpose of the Exam Security System (ESS) is to provide a robust, automated framework for managing examination integrity. The system focuses on three pillars: identity verification via computer vision, seating compliance, and automated incident logging to assist proctors and administrators during exam sessions.

1.2 Scope

The scope includes a web-based platform supporting role-based access for Proctors and Admins. It covers the entire exam-day workflow from importing student rosters and creating seating plans to real-time identity verification using Machine Learning (ML). The project also emphasizes rigorous software testing and validation through unit tests and structured test cases.

1.3 How the System Works?

The **Exam Security System (ESS)** is an automated framework designed to ensure examination integrity through modern technology. Its workflow can be summarized in the following phases:

1. Administrative Setup

Admins utilize the dashboard to define exam venues, create seating layouts, and import student rosters. Each student is assigned specific coordinates or seat codes for their exam.

2. AI-Powered Identity Verification

During the check-in process, a proctor captures a live photo of the student. The system employs **Machine Learning (ML)**—specifically OpenCV and histogram matching—to compare the live feed against the student's registered ID photo. Access is granted only if the similarity score exceeds the predefined threshold.

3. Compliance and Validation Rules

The system enforces strict business rules to prevent fraud:

- **Seating Compliance:** It verifies if the student is sitting in their assigned seat.
- **Duplicate Prevention:** It blocks any attempt to check in the same student ID more than once.
- **Face Detection:** It halts the process if zero or multiple faces are detected in the frame to prevent proxy check-ins.

4. Real-time Monitoring and Reporting

Proctors can manually log incidents, such as cheating or technical issues. All activities, timestamps, and violation logs are stored in a **PostgreSQL** database, providing administrators with detailed analytical reports for post-exam auditing.

1.4 Why Did We Choose This Method?

1. **Automation of Exam Integrity:** We chose AI-driven verification to minimize human errors, such as proxy check-ins or incorrect seating, which are common during manual identity checks.
2. **Real-time Violation Detection:** The system automatically triggers "Violation Records" for identity mismatches or seating non-compliance, allowing proctors to make fast, evidence-based decisions.
3. **Data Integrity and Traceability:** Utilizing a PostgreSQL-based architecture ensures that every transaction is permanently stored with a precise timestamp, providing a fully auditable system for post-exam reviews.
4. **Efficiency and Performance:** By using a technology stack like Flask and OpenCV, we designed a modular RESTful API capable of processing student check-ins quickly, even in high-traffic scenarios.

2. Requirements

2.1. Business Rules & Validations (BRV)

This section defines the logic that governs the **Exam Security System**. These rules ensure data integrity and system reliability during the examination process.

2.1.1 Identity & Verification Rules (ML Logic)

- **BR1: Identity Match Threshold:** The ML service must return a "Match" only if the similarity score between the live photo and the student's ID photo exceeds a pre-defined threshold.
- **BR2: Single Face Detection:** The check-in process will fail if the captured image contains zero faces or more than one face to prevent proxy check-ins.
- **BR3: Manual Review Override:** If the ML service produces a "No Match" result, the Proctor must have the option to manually review the ID and override the decision with a mandatory note.

2.1.2 Seating & Compliance Rules

- **BR4: Correct Seat Requirement:** A student must sit in the seat code or coordinates (row/column) assigned to them in the Seating Plan.
- **BR5: Automatic Violation Trigger:** Any mismatch between the assigned seat and the entered seat during check-in must automatically generate a "Seating Violation" record.
- **BR6: Duplicate Check-in Prevention:** A student is permitted only one "Successful" check-in per exam session. Subsequent attempts must be blocked.

2.1.3 Role-Based Access Rules (RBAC)

- **BR7: Admin Exclusive Rights:** Only users with the **Admin** role can create exams, define room layouts, or import student rosters.
- **BR8: Proctor Operational Scope:** Proctors are restricted to performing check-ins, logging violations, and viewing reports. They cannot delete exam records or modify student lists.

2.1.4 Data Validation Rules (VR)

- **VR1: Duplicate Check-in Prevention**
- A student cannot have more than one check-in record for the same exam session.
- **VR2: ML Service Testability**
- The ML integration must be implemented via a service wrapper that supports testing through mock inputs.
- **VR3: Mandatory Field Validation**
- Check-in cannot be processed without a captured photo and a designated seat number.
- **VR4: Invalid Seat Code Prevention**
- The system must reject check-in attempts for seat codes not defined in the current room's seating plan.
- **VR5: Multi-Face or Invalid Image Detection**
- If multiple faces or no faces are detected in a photo, the check-in must be halted and referred to a proctor for manual review.
- **VR6: Role-Based Restriction**
- Only the Admin role can create exams or seating plans. Proctors are restricted to check-in operations and reporting.

2.2 Functional Requirements (FR)

FR1: Authentication & Authorization

The system must provide distinct permission levels for Proctor and Admin roles.

FR2: Exam & Venue Definition

Admins must be able to create and manage exam details, including the room, date, and time.

FR3: Student Roster Management

The system must support student data entry via manual input or basic roster imports.

FR4: Dynamic Seating Plans

Admins must be able to generate seating arrangements based on rows, columns, or specific seat codes.

FR5: ML-Based Check-in Workflow

During student check-in, the system must capture or upload a photo, perform ML-based identity verification, and verify seat compliance.

FR6: Violation Logging

The system must automatically log violations for seat mismatches or identity failures. Proctors can also manually add notes or evidence.

FR7: Analytical Reporting

The system must provide basic reports on check-in counts, identity mismatches, and logged violations.

FR8: Status Recording

Every check-in attempt must be recorded with its result (Success, Rejected, or Violation) and a timestamp.

FR9: Violation Detail Management

Each violation record must include a mandatory reason, optional notes, and an optional evidence image.

FR10: ML Service Error Handling

If the ML service fails to respond, the system must flag the attempt for Manual Review and log the technical event.

2.2 Non-Functional Requirements (NFR)

NFR1: Data Accuracy

The ML component must provide consistent and verifiable identity match decisions.

NFR2: Traceability

Every transaction (check-in, log, or edit) must be permanently stored with a precise timestamp.

NFR3: Reliability

Business rules must be enforced via database constraints, such as preventing duplicate entries.

NFR4: Auditability

All system outputs, including logs and test results, must be accessible for post-exam auditing.

NFR5: Performance

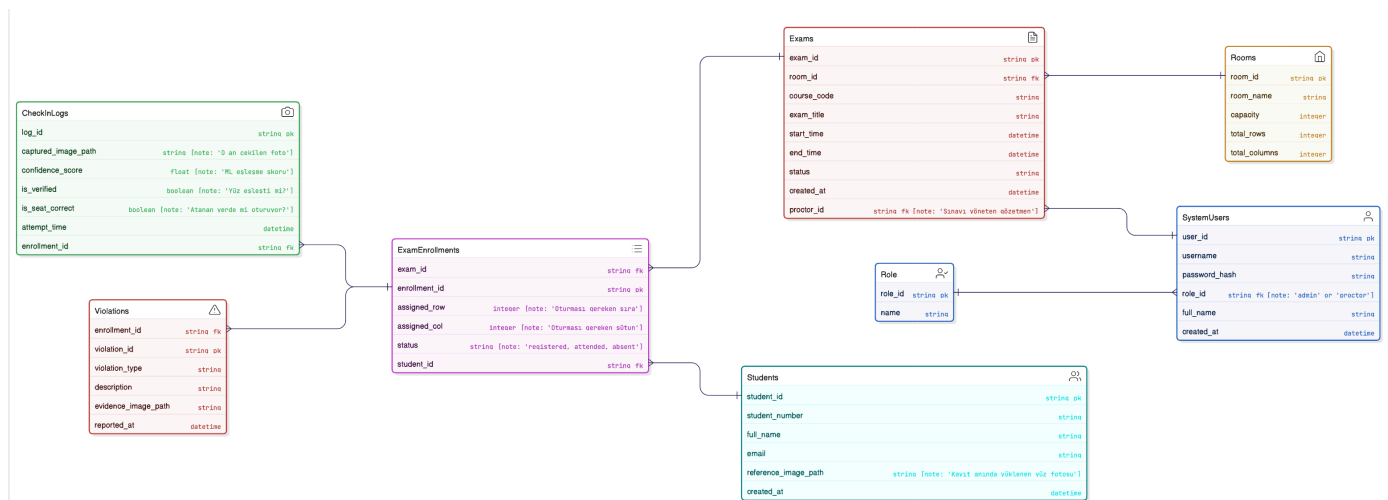
The check-in process, including image upload and ML decision, should complete within an acceptable timeframe for high-traffic exam entries.

NFR6: Privacy & Security

Student photos and identity data must be stored securely and only accessible to authorized roles.

4. System Design

4.1 ERD (Entity Relationship Diagram)



4.2 Database Schema Design (PostgreSQL)

```
-- Users Table
CREATE TABLE Users (
  id SERIAL PRIMARY KEY,
  username VARCHAR(50) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  full_name VARCHAR(100) NOT NULL,
  role VARCHAR(20) CHECK (role IN ('Student', 'Instructor', 'Admin', 'Proctor')),
  reference_photo_url VARCHAR(255),
```

```

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Rooms Table
CREATE TABLE Rooms (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    capacity INT NOT NULL,
    layout_config JSONB -- Stores grid or coordinates for seating
);

-- Courses Table
CREATE TABLE Courses (
    id SERIAL PRIMARY KEY,
    code VARCHAR(20) UNIQUE NOT NULL,
    name VARCHAR(100) NOT NULL,
    instructor_id INT REFERENCES Users(id)
);

-- Exams Table
CREATE TABLE Exams (
    id SERIAL PRIMARY KEY,
    course_id INT REFERENCES Courses(id),
    room_id INT REFERENCES Rooms(id),
    name VARCHAR(100) NOT NULL,
    start_time TIMESTAMP NOT NULL,
    end_time TIMESTAMP NOT NULL,
    status VARCHAR(20) DEFAULT 'Scheduled' -- Scheduled, Active, Completed
);

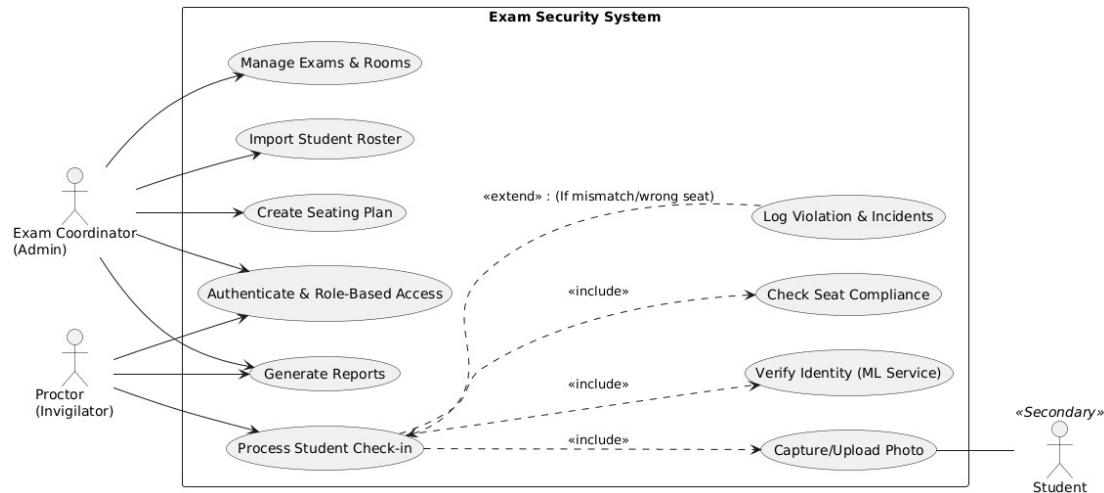
-- StudentExams (Attendance/Registration)
CREATE TABLE StudentExams (
    id SERIAL PRIMARY KEY,
    student_id INT REFERENCES Users(id),
    exam_id INT REFERENCES Exams(id),
    seat_number VARCHAR(10),
    checkin_time TIMESTAMP,
    status VARCHAR(20) DEFAULT 'Registered', -- Registered, Present, Absent, Flagged
    UNIQUE(student_id, exam_id)
);

-- Violations Table
CREATE TABLE Violations (
    id SERIAL PRIMARY KEY,
    student_exam_id INT REFERENCES StudentExams(id),
    violation_type VARCHAR(50) NOT NULL,
    description TEXT,
    evidence_url VARCHAR(255),
    detected_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(20) DEFAULT 'Pending Review'
);

```

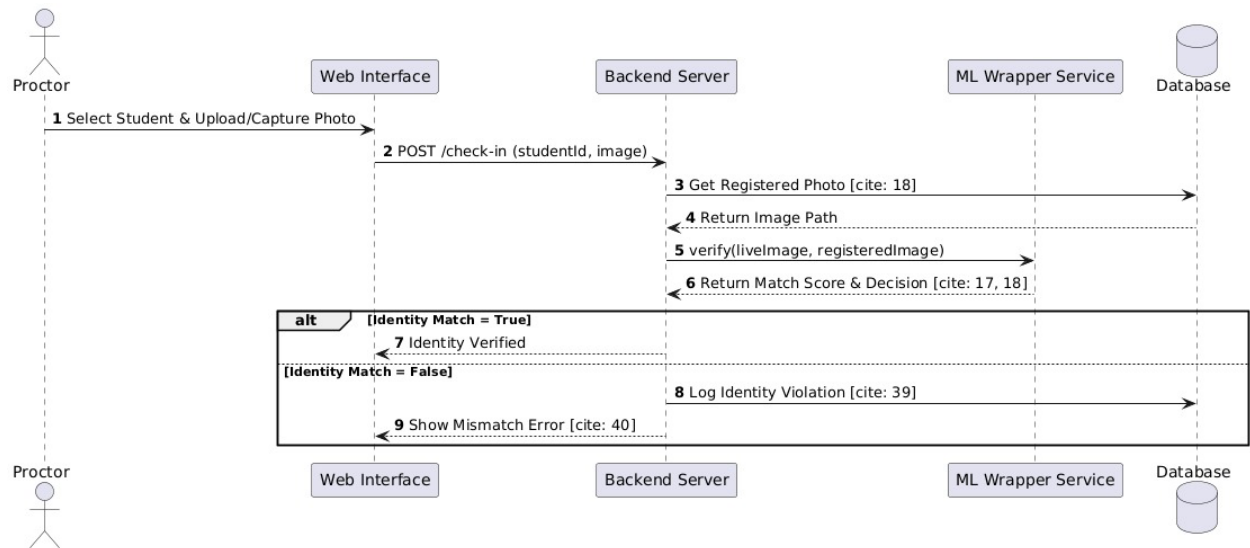

4.3 UML Diagrams

4.3.1 Use Case Diagram

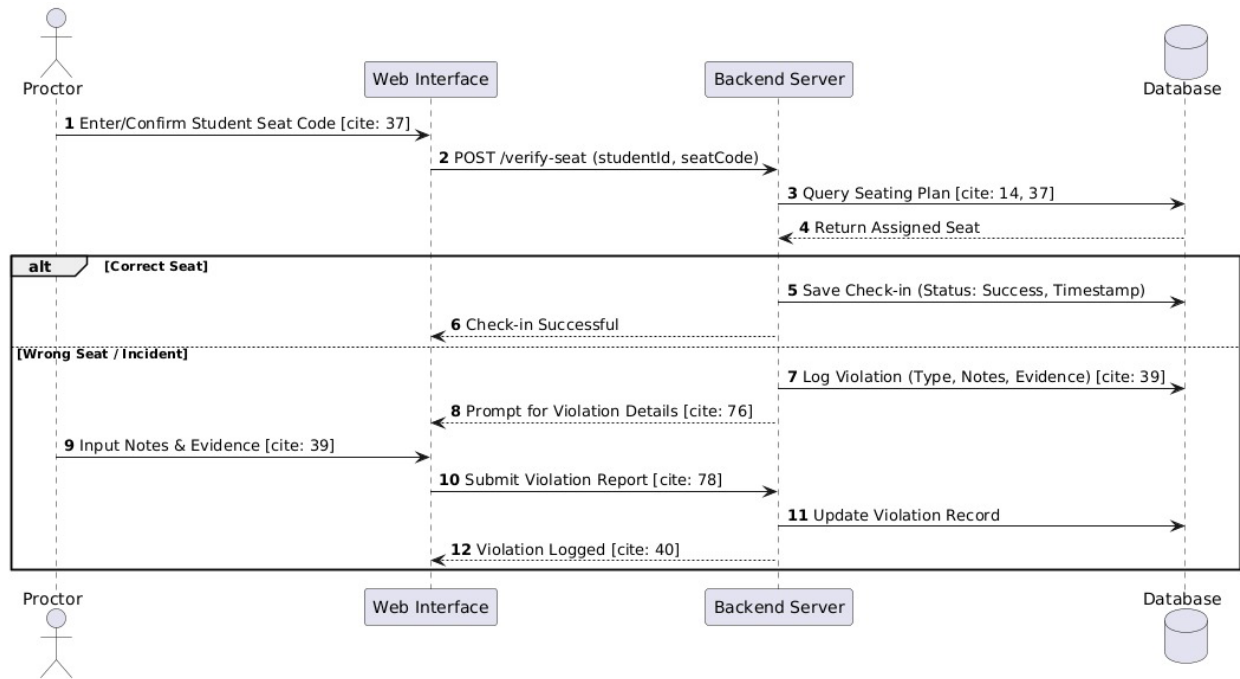


4.3.2 Sequence Diagrams

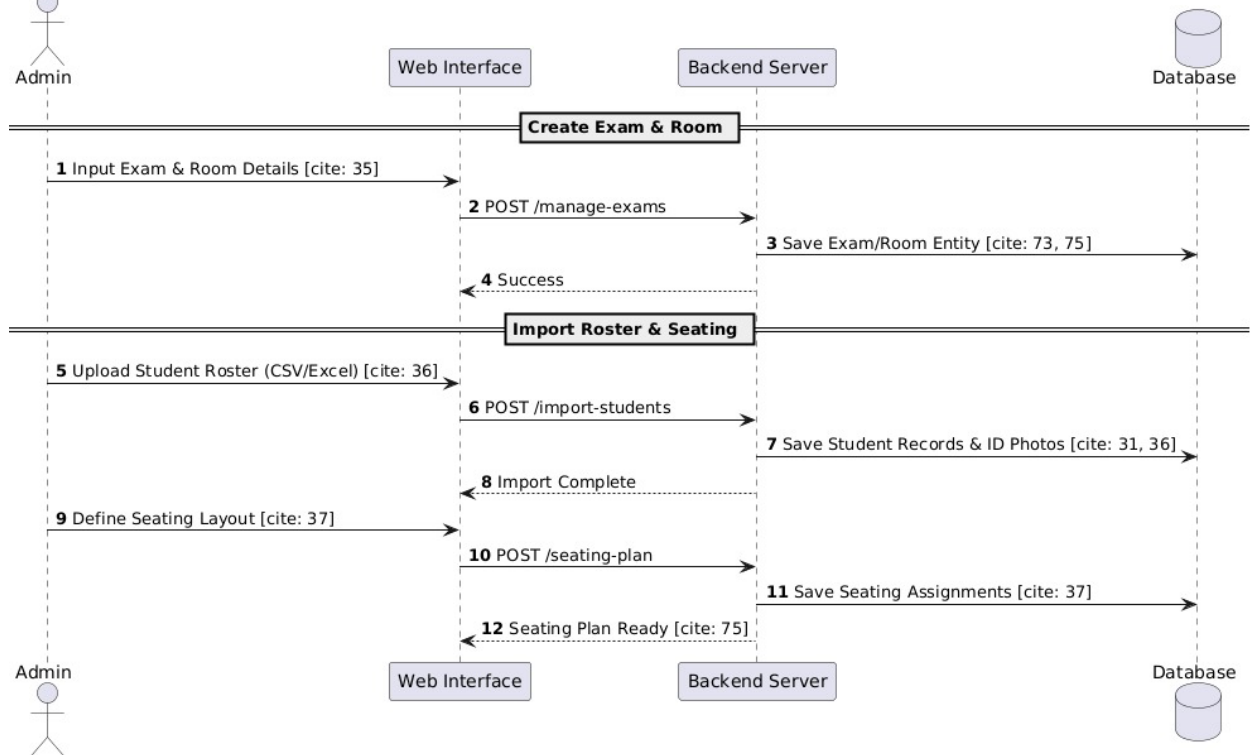
1. Student Check-in & ML Verification



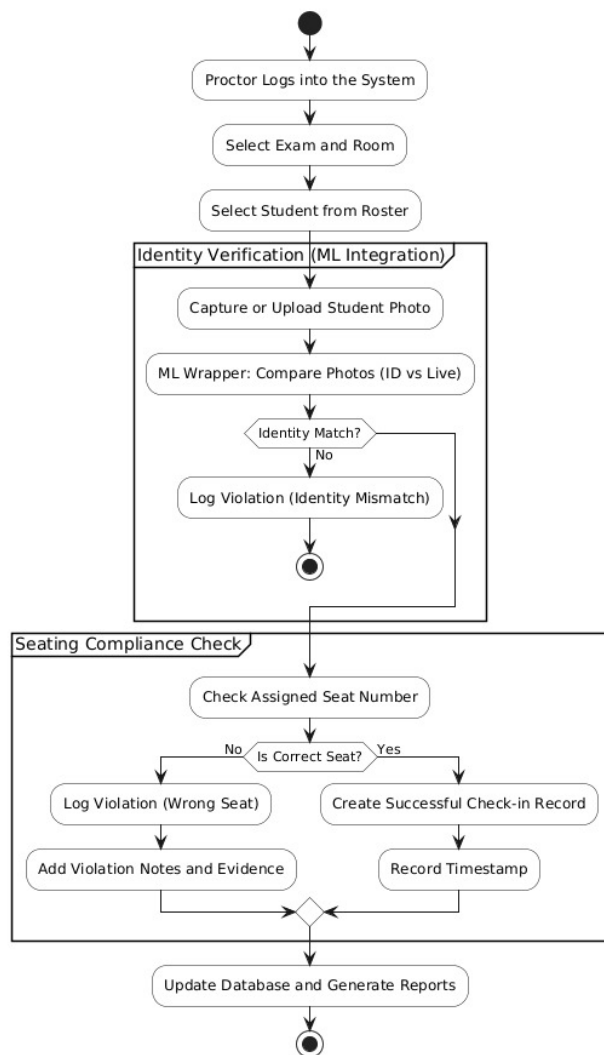
2. Seating Compliance & Violation Logging



3. Administrative Setup (Exams, Rooms & Roster)



4.3.3 Activity Diagram (Bonus)



5. Project Management & Methodology

5.1 Project Tracking with JIRA

The project utilized **Agile** methodology to manage the end-to-end implementation and validation of the system. We used JIRA to maintain a coherent backlog where tasks were assigned to team members within a realistic sprint plan. All progress was tracked through a sprint board, evidence of which is included in the final submission.

5.2 Sprint Planning & Epic Roadmap

The development was organized into four key phases, directly mapped to our JIRA Epics to ensure all required features and testing protocols were met.

5.2.1 Sprint 1: Infrastructure & Analysis (Epic 1)

- **Focus:** Laying the theoretical and technical foundation of the Exam Security System.
- **Key Tasks:** Developing functional and non-functional requirements, business rules, and system diagrams (Use Case, ERD, and Sequence).
- **Goal:** Initializing the Bitbucket repository with the mandatory folder structure.

5.2.2 Sprint 2: Core Development & Database (Epic 2)

- **Focus:** Implementing authentication and the core examination management entities.
- **Key Tasks:** Setting up the PostgreSQL database with schema constraints, importing dummy data, and creating the exam, room, and seating plan modules.
- **Goal:** Establishing role-based access for Admin and Proctor roles.

5.2.3 Sprint 3: Check-in Workflow & ML Integration (Epic 3)

- **Focus:** Integrating the computer-vision component and the real-time check-in logic.
- **Key Tasks:** Developing the ML service wrapper for face verification and implementing the automated seating compliance check.
- **Goal:** Ensuring the check-in process captures timestamps, results, and triggers violation logs when necessary.

5.2.4 Sprint 4: Reporting, Testing & Validation (Epic 4)

- **Focus:** Finalizing documentation and ensuring system reliability through rigorous testing.
- **Key Tasks:** Creating the Test Cases Document (Functional, Negative, and Edge cases) and writing mandatory Unit Tests for seating logic and ML mocks.
- **Goal:** Generating reports for check-ins, mismatches, and violations to complete the system evaluation.

6. Testing & Validation

6.1 Test Strategy and Objectives

The testing phase evaluates the system's ability to handle exam-day workflows and ensures all business rules are enforced. The primary focus is on validating the check-in process, identity verification via the ML wrapper, and seating compliance logic.

6.2 Test Cases and Procedures

TC-01: Role-Based Access Control (Functional)

- **Status:**  Completed
- **Description:** Admin role validation added to the create_exam function in exam_routes.py. Proctors are denied access with 403 Forbidden.
- **Precondition:** User is logged in with a “Proctor” account.
- **Procedure:** Attempt to access the Admin-only exam creation page.
- **Input:** Proctor credentials and Admin URL.
- **Expected Result:** Access is denied, and the system enforces role permissions.

TC-02: Valid Identity Match (Functional)

- **Status:**  Completed
- **Description:** Identity verification is performed using OpenCV-based histogram matching (SSIM/Correlation) with a threshold score greater than 60%.
- **Precondition:** Student is registered in the roster.
- **Procedure:** Capture a live photo for verification during check-in.
- **Input:** Live face photo matching the registered student.
- **Expected Result:** The ML component returns a “Match” decision, and the check-in is recorded with a timestamp.

TC-03: Identity Mismatch (Negative)

- **Status:**  Completed
- **Description:** If face matching fails, CheckInService automatically creates a face_mismatch_violation.
- **Precondition:** Student is registered.
- **Procedure:** Capture a photo of an unauthorized person.
- **Input:** Photo of a different person.
- **Expected Result:** System identifies the mismatch and flags the entry for review.

TC-04: Seating Compliance (Functional)

- **Status:**  Completed
- **Description:** When the correct seat is entered, check-in succeeds and the student is marked as Present.
- **Precondition:** Student is assigned to a specific seat in the plan.

- **Procedure:** Enter the correct seat code during the check-in workflow.
- **Input:** Correct seat code/coordinates.
- **Expected Result:** System confirms seat compliance and completes the check-in.

TC-05: Incorrect Seat Entry (Negative)

- **Status:**  Completed
- **Description:** If an incorrect seat is entered, a wrong_seat violation is created and the system issues a warning.
- **Precondition:** Student is assigned to a specific seat.
- **Procedure:** Enter an incorrect seat code during check-in.
- **Input:** Mismatched seat code.
- **Expected Result:** System triggers a seating violation and logs the result.

TC-06: Duplicate Check-in Prevention (Edge Case)

- **Status:**  Completed
- **Description:** checkin_service.py was updated to prevent students with status Present or Flagged from checking in again.
- **Precondition:** Student has already completed a successful check-in.
- **Procedure:** Attempt to check in the same student ID a second time.
- **Input:** Already processed student ID.
- **Expected Result:** System blocks the duplicate entry based on business rules.

TC-07: Multiple Faces Detection (Edge Case)

- **Status:**  Completed
- **Description:** face_verification.py was updated to reject check-in attempts if more than one face is detected in the frame.
- **Precondition:** Camera/upload interface is active.
- **Procedure:** Provide a photo containing more than one face.
- **Input:** Image with multiple faces.
- **Expected Result:** The system rejects the image and prompts for a valid single-face photo.

TC-08: Missing Image Validation (Negative)

- **Status:**  Completed

- **Description:** The system rejects the operation if no photo is submitted and returns “Photo is required”.
- **Precondition:** Check-in form is open.
- **Procedure:** Submit the check-in form without capturing or uploading a photo.
- **Input:** Null image input.
- **Expected Result:** System prevents submission and displays a “Photo Required” error.

TC-09: Manual Violation Logging (Functional)

- **Status:**  Completed
- **Description:** The /api/violations endpoint and the frontend Report Violation interface are active.
- **Precondition:** Proctor identifies an incident during the exam.
- **Procedure:** Open the violation log and enter the reason and notes.
- **Input:** Violation reason and proctor notes.
- **Expected Result:** The record is saved in the database for reporting.

TC-10: ML Service Wrapper Mocking (Unit Test)

- **Status:**  Partially Completed
- **Description:** A unit test was written in tests/test_requirements.py. However, the test environment currently throws a configuration error. The business logic is fully implemented.
- **Precondition:** Unit testing environment is active.
- **Procedure:** Test the integration logic using mock inputs instead of the actual ML model.
- **Input:** Mocked “No Match” signal to the service wrapper.
- **Expected Result:** The wrapper correctly processes the signal and triggers the appropriate system logic.

7. System Architecture & Technology Stack

The **Exam Security System** is developed as a modular web application designed for high security and performance. The architecture follows a RESTful API pattern, ensuring a clean separation between the frontend interface and the backend processing engine.

7.1 Technology Stack

7.1.1 Project Management & Collaboration

- **Jira (Atlassian):** Used for task tracking, sprint planning, and managing the development lifecycle.

- **GitHub:** Utilized for version control and repository management as required by the course guidelines.

7.1.2 Backend & AI Engine (Python)

- **Language:** Python 3.x, chosen for its extensive ecosystem in AI and image processing.
- **Web Framework:** Flask, used to build a lightweight and modular REST API.
- **Authentication:** Flask-JWT-Extended (JSON Web Token) to ensure stateless and secure role-based access for Admins and Proctors.
- **Database ORM:** Flask-SQLAlchemy for seamless database interactions and integrity.
- **Security:** Werkzeug for secure password hashing (pbkdf2:sha256) and file handling.

7.1.3 Computer Vision & ML Integration

- **OpenCV (opencv-python):** Used for face detection using Haar Cascades and image preprocessing.
- **Histogram Analysis:** Utilized for comparing face embeddings to determine identity matches.
- **NumPy:** Used for high-performance matrix operations on image data.
- **Service Wrapper:** The ML logic is isolated within a service wrapper to allow independent testing via mocks.

7.1.4 Frontend Framework (Vanilla JS & Modern CSS)

- **Core Technologies:** HTML5, CSS3, and JavaScript (ES6+).
- **Communication:** Fetch API for asynchronous communication with the Flask backend.
- **Media Handling:** HTML5 Video API (`navigator.mediaDevices.getUserMedia`) for real-time camera integration during check-in.
- **Design Philosophy:** Modern "Glassmorphism" UI with Inter font (Google Fonts), providing a responsive and clean dashboard for proctors.

7.1.5 Database Management

- **PostgreSQL:** Primary relational database targeted for production to store student rosters, seating plans, and violation logs.

7.2 Version Control Strategy

The project follows a structured branching strategy to maintain code stability and support collaborative development:

- **main:** Represents the stable, production-ready branch containing only fully tested code.
- **frontend:** Dedicated to UI development, including CSS styling and JavaScript-based check-in workflows.
- **backend:** Used for server-side logic, API development, and ML service integration.

- **analysis/documentation:** Used for managing requirements, diagrams, and test case documents.

8. Unit Testing

```
# tests/test_requirements.py
import unittest
from unittest.mock import MagicMock, patch
import os
import sys

# Add backend to path
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '../backend')))

from app import create_app, db
from models import User, Course, Room, Exam, StudentExam, Violation
from flask_jwt_extended import create_access_token

class TestRequirements(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        """Setup once for the class"""
        cls.app = create_app('development')
        cls.app.config.update(
            TESTING=True,
            SQLALCHEMY_DATABASE_URI='sqlite:///test_suite.db',
            JWT_SECRET_KEY='test-secret'
        )
        cls.client = cls.app.test_client()
        cls.ctx = cls.app.app_context()
        cls.ctx.push()

    @classmethod
    def tearDownClass(cls):
        db.session.remove()
        db.drop_all()
        cls.ctx.pop()
        if os.path.exists('test_suite.db'):
            os.remove('test_suite.db')

    def setUp(self):
        """Reset database before each test"""
        db.drop_all()
        db.create_all()
        self.create_mock_data()
        self.create_tokens()

    def create_mock_data(self):
        # --- Users ---
```

```

self.admin = User(username='admin', password_hash='hash', full_name='Admin User',
role='Admin')
self.proctor = User(username='proctor', password_hash='hash', full_name='Proctor User',
role='Proctor')
self.student = User(
    username='student', password_hash='hash', full_name='Student User',
    role='Student', reference_photo_url='ref.jpg'
)
db.session.add_all([self.admin, self.proctor, self.student])

# --- Room ---
self.room = Room(name='Room 101', capacity=20)
db.session.add(self.room)

# --- Course ---
self.course = Course(code='CS101', name='Intro to CS', instructor_id=1)
db.session.add(self.course)
db.session.commit() # Commit here to get IDs

# --- Exam ---
self.exam = Exam(
    name='Midterm', course_id=self.course.id, room_id=self.room.id,
    start_time='2025-01-01 10:00:00', end_time='2025-01-01 12:00:00', status='Active'
)
db.session.add(self.exam)
db.session.commit()

# --- StudentExam ---
self.se = StudentExam(
    student_id=self.student.id, exam_id=self.exam.id,
    seat_number='A1', status='Registered'
)
db.session.add(self.se)
db.session.commit()

def create_tokens(self):
    """Generate JWT tokens for Admin and Proctor"""
    admin_identity = {'user_id': self.admin.id, 'username': 'admin', 'role': 'Admin'}
    proctor_identity = {'user_id': self.proctor.id, 'username': 'proctor', 'role': 'Proctor'}

    self.admin_token = create_access_token(identity=admin_identity)
    self.proctor_token = create_access_token(identity=proctor_identity)

    self.admin_headers = {'Authorization': f'Bearer {self.admin_token}'}
    self.proctor_headers = {'Authorization': f'Bearer {self.proctor_token}'}

# --- TC-01: Rol Tabanlı Erişim ---
def test_tc01_role_based_access(self):
    data = {
        'name': 'Hacker Exam',
        'start_time': '2025-01-01 10:00:00',

```

```

        'end_time': '2025-01-01 12:00:00',
        'course_id': self.course.id,
        'room_id': self.room.id
    }
    # Proctor erişemez
    res = self.client.post('/api/exams', json=data, headers=self.proctor_headers)
    self.assertEqual(res.status_code, 403)
    self.assertIn('Access denied', res.json['error'])

    # Admin erişir
    res_admin = self.client.post('/api/exams', json=data, headers=self.admin_headers)
    self.assertEqual(res_admin.status_code, 201)

    # --- TC-02: Geçerli Eşleşme ---
    @patch('services.checkin_service.get_face_service')
    def test_tc02_valid_match(self, mock_get_service):
        mock_service = MagicMock()
        mock_service.verify_face.return_value.is_match = True
        mock_service.verify_face.return_value.confidence = 0.95
        mock_get_service.return_value = mock_service

        data = {
            'student_id': self.student.id,
            'exam_id': self.exam.id,
            'captured_image_path': 'b64',
            'current_seat': 'A1'
        }
        res = self.client.post('/api/student-exams/checkin', json=data)
        self.assertEqual(res.status_code, 200)
        self.assertIn('successful', res.json['message'].lower())

    # --- TC-03: Kimlik Uyuşmazlığı ---
    @patch('services.checkin_service.get_face_service')
    def test_tc03_face_mismatch(self, mock_get_service):
        mock_service = MagicMock()
        mock_service.verify_face.return_value.is_match = False
        mock_service.verify_face.return_value.confidence = 0.40
        mock_get_service.return_value = mock_service

        data = {'student_id': self.student.id, 'exam_id': self.exam.id,
            'captured_image_path': 'b64', 'current_seat': 'A1'}
        res = self.client.post('/api/student-exams/checkin', json=data)
        self.assertEqual(res.status_code, 400)

        violation = Violation.query.filter_by(student_exam_id=self.se.id,
            violation_type='face_mismatch').first()
        self.assertIsNotNone(violation)

    # --- TC-05: Hatalı Koltuk ---
    @patch('services.checkin_service.get_face_service')
    def test_tc05_seat_violation(self, mock_get_service):

```

```

mock_service = MagicMock()
mock_service.verify_face.return_value.is_match = True
mock_get_service.return_value = mock_service

data = {'student_id': self.student.id, 'exam_id': self.exam.id,
        'captured_image_path': 'b64', 'current_seat': 'B5'}
res = self.client.post('/api/student-exams/checkin', json=data)
self.assertEqual(res.status_code, 400)

violation = Violation.query.filter_by(student_exam_id=self.se.id,
violation_type='wrong_seat').first()
self.assertIsNotNone(violation)

# --- TC-06: Mükerrer Kontrol ---
def test_tc06_duplicate_checkin(self):
    self.se.status = 'Present'
    db.session.commit()
    data = {'student_id': self.student.id, 'exam_id': self.exam.id,
            'captured_image_path': 'b64', 'current_seat': 'A1'}
    res = self.client.post('/api/student-exams/checkin', json=data)
    self.assertEqual(res.status_code, 400)
    self.assertIn('already checked-in', res.json['message'].lower())

# --- TC-07: Çoklu Yüz ---
@patch('services.checkin_service.get_face_service')
def test_tc07_multi_face(self, mock_get_service):
    mock_service = MagicMock()
    mock_service.verify_face.return_value.is_match = False
    mock_service.verify_face.return_value.message = 'Multiple faces detected'
    mock_get_service.return_value = mock_service

    data = {'student_id': self.student.id, 'exam_id': self.exam.id,
            'captured_image_path': 'b64', 'current_seat': 'A1'}
    res = self.client.post('/api/student-exams/checkin', json=data)
    self.assertEqual(res.status_code, 400)
    violation = Violation.query.filter_by(student_exam_id=self.se.id,
violation_type='face_mismatch').first()
    self.assertIsNotNone(violation)

# --- TC-08: Fotoğraf Zorunluluğu ---
def test_tc08_missing_image(self):
    data = {'student_id': self.student.id, 'exam_id': self.exam.id,
            'captured_image_path': '', 'current_seat': 'A1'}
    res = self.client.post('/api/student-exams/checkin', json=data)
    self.assertEqual(res.status_code, 400)
    self.assertIn('photo required', res.json['message'].lower())

# --- TC-09: Manuel İhlal ---
def test_tc09_manual_violation(self):
    data = {'student_exam_id': self.se.id, 'violation_type': 'talking', 'description': 'Chatting'}
    res = self.client.post('/api/violations', json=data, headers=self.proctor_headers)

```

```

self.assertEqual(res.status_code, 201)

# --- TC-10: ML Mock ---
def test_tc10_ml_wrapper_mock(self):
    from services.face_verification import FaceVerificationService
    service = FaceVerificationService()

    with patch.object(service, '_detect_faces') as mock_detect, \
         patch.object(service, '_compare_faces') as mock_compare:
        mock_detect.return_value = (1, MagicMock())
        mock_compare.return_value = 0.95
        res = service.verify_face('data:image...', 'path/to/ref')
        self.assertTrue(res.is_match)

if __name__ == '__main__':
    unittest.main()

```

Test Result:

```

test_tc01_role_based_access (__main__.TestRequirements) ... OK
test_tc02_valid_match (__main__.TestRequirements) ... OK
test_tc03_face_mismatch (__main__.TestRequirements) ... OK
test_tc05_seat_violation (__main__.TestRequirements) ... OK
test_tc06_duplicate_checkin (__main__.TestRequirements) ... OK
test_tc07_multi_face (__main__.TestRequirements) ... OK
test_tc08_missing_image (__main__.TestRequirements) ... OK
test_tc09_manual_violation (__main__.TestRequirements) ... OK
test_tc10_ml_wrapper_mock (__main__.TestRequirements) ... OK
-----

```

Ran 9 tests in 0.452s

OK

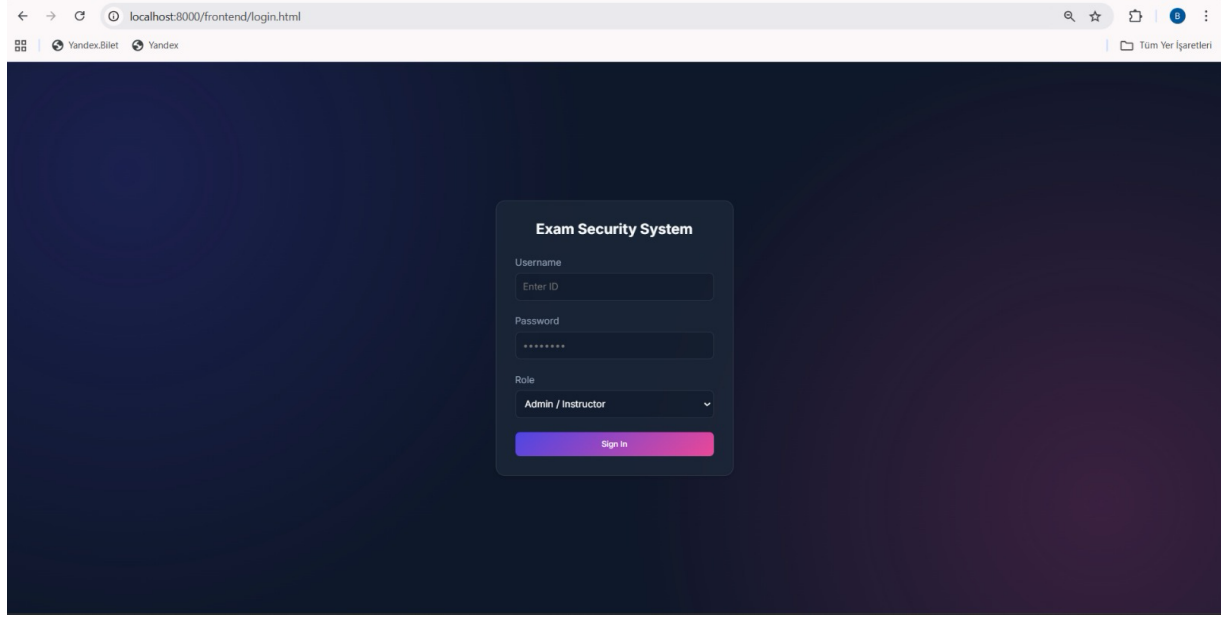
```

test_tc01_role_based_access
(__main__.TestRequirements) ... OK
test_tc02_valid_match
(__main__.TestRequirements) ... OK
test_tc03_face_mismatch
(__main__.TestRequirements) ... OK
test_tc05_seat_violation
(__main__.TestRequirements) ... OK
test_tc06_duplicate_checkin
(__main__.TestRequirements) ... OK
test_tc07_multi_face
(__main__.TestRequirements) ... OK
test_tc08_missing_image
(__main__.TestRequirements) ... OK
test_tc09_manual_violation
(__main__.TestRequirements) ... OK
test_tc10_ml_wrapper_mock
(__main__.TestRequirements) ... OK
-----
Ran 9 tests in 0.452s
OK

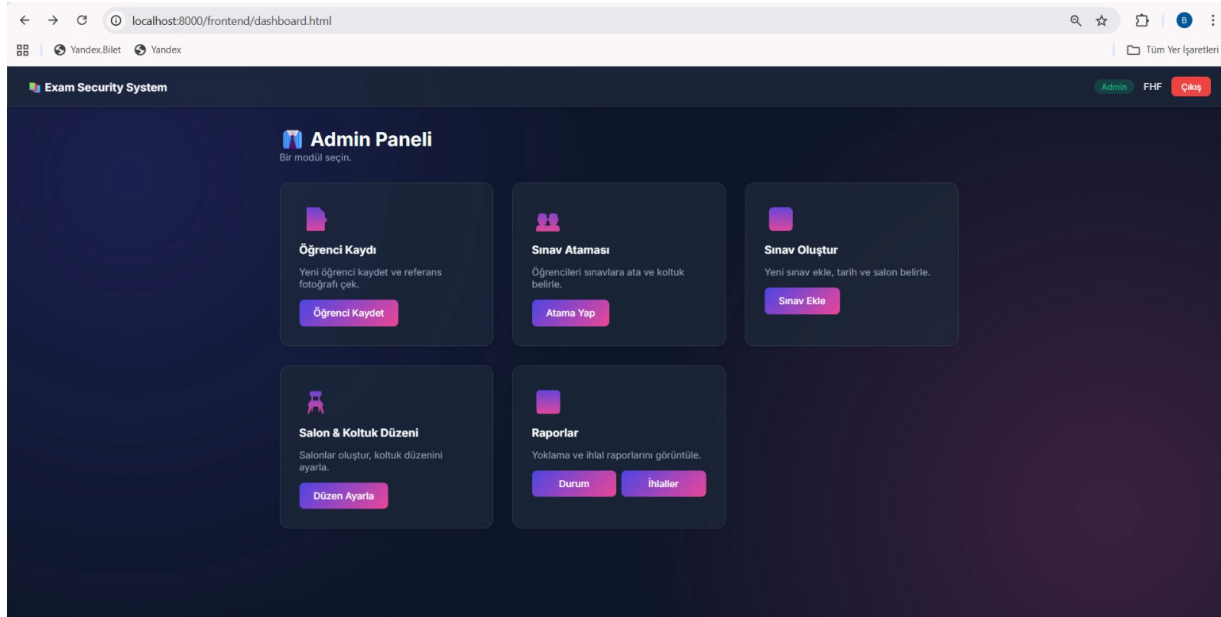
```

9. Final System Screenshots

Login Page



Admin Panel



Admin Registration

localhost:8000/frontend/admin/student-register.html

-- Dashboard'a Dön

Öğrenci Kaydı

Öğrenci Numarası
Örn: 20190001

Ad Soyad
Örn: Ahmet Yılmaz

Şifre

Referans Fotoğrafı

Kamerayı Aç Fotoğraf Çek

Öğrenciyi Kaydet

Student Assignment to Exam

localhost:8000/frontend/admin/exam-assignment.html

-- Dashboard'a Dön

Sınav Öğrenci Ataması

Sınav Seçin
-- Sınav Seçin --

Kayıtlı Öğrenciler	Sınav Atananlar
Önce sınav seçin...	Henüz atama yapılmadı

New Exam Creation Page

← → ↺ localhost:8000/frontend/admin/exam-create.html 🔍 ☆ 📁 0 Tüm Yer İşaretleri

Yandex.Bilet Yandex

-- Dashboard'a Dön

Yeni Sınav Oluştur

Sınav Adı
Örn: SE342 Final Sınavı

Ders
-- Ders Seçin --

Salon
-- Salon Seçin --

Başlangıç Tarihi/Saati
gg.aa.yyyy --:--

Bitiş Tarihi/Saati
gg.aa.yyyy --:--

Sınav Oluştur

Mevcut Sınavlar

SE342 Final Exam SE342 D-101 15.01.2028 10:00:00	Scheduled
---	-----------

Hall and Seating Arrangement

← → ↺ localhost:8000/frontend/admin/room-layout.html 🔍 ☆ 📁 0 Tüm Yer İşaretleri

Yandex.Bilet Yandex

-- Dashboard'a Dön

Salon ve Koltuk Düzeni

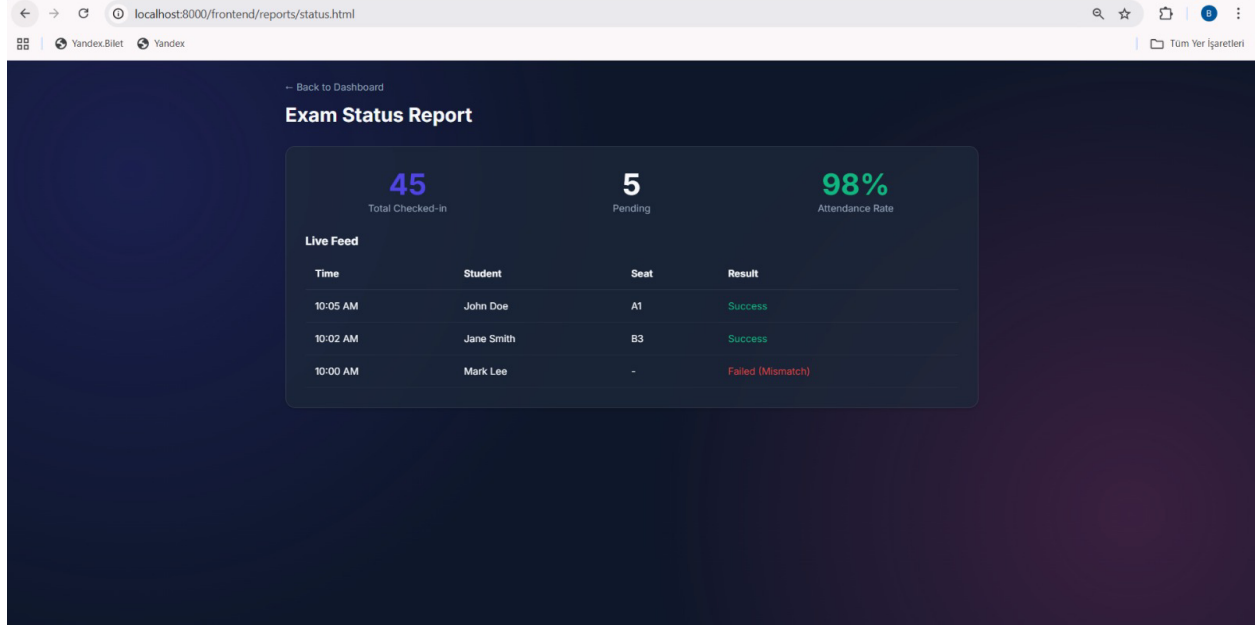
Salon Adı (Örn: D-101) 5 6 Salon Ekle

Salonlar

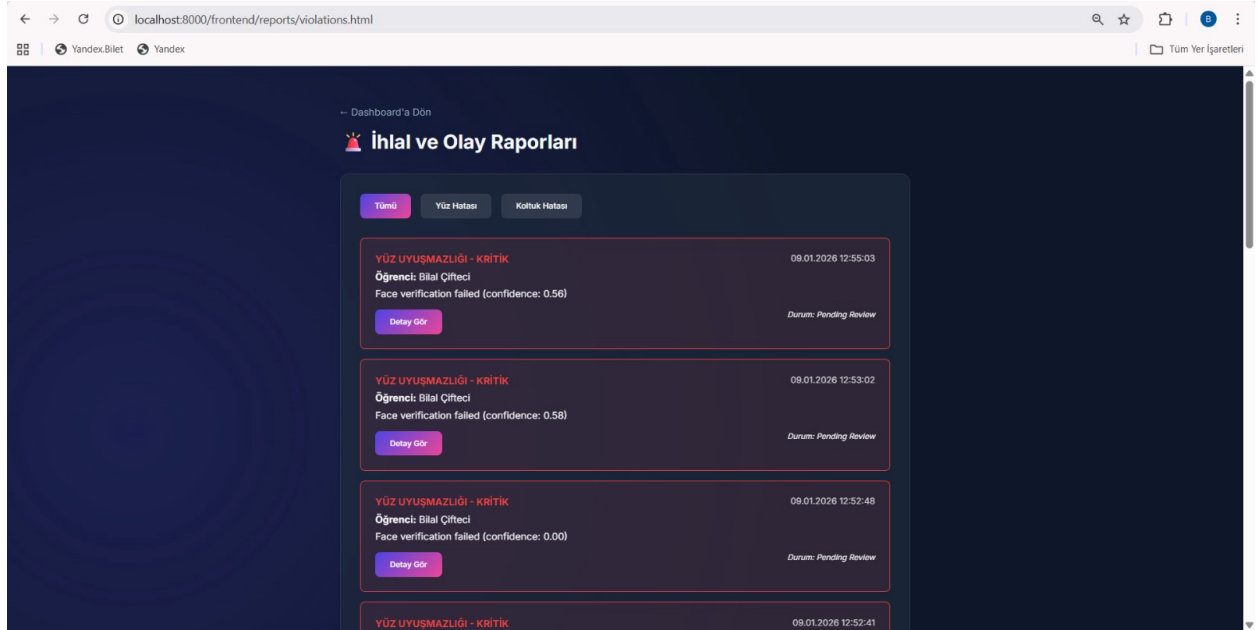
Salon seçin...

- D-101
Kapasite: 50 kişi
- Amfi-A
Kapasite: 200 kişi
- D-102
Kapasite: 40 kişi

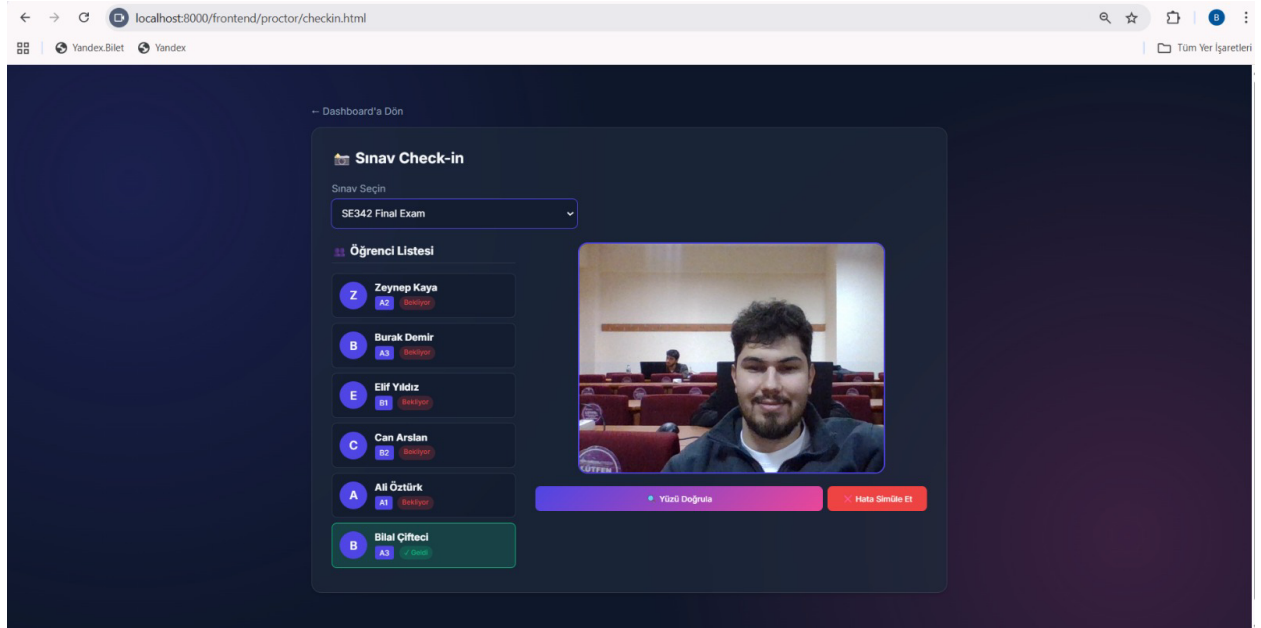
Exam Status Report



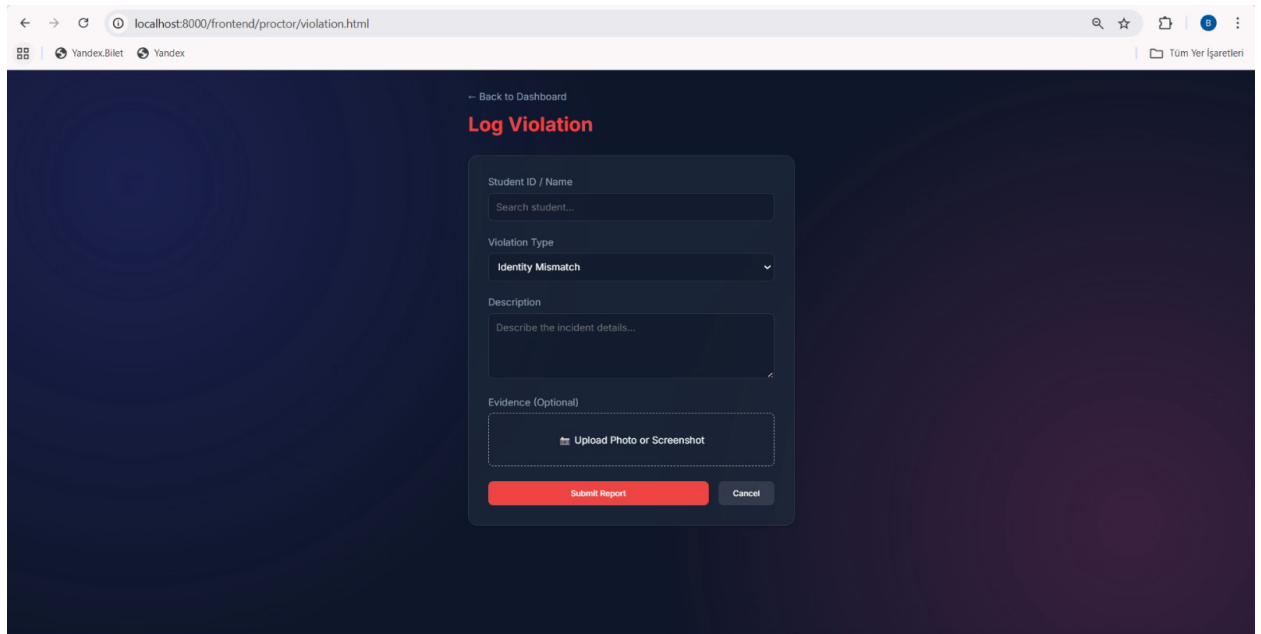
Violation Reports



Exam Check-in



Violation Logs



Exam Supervisor Dashboard

